

Low-Cost UMI Report

A Wireless Bimanual Data Collection System for Imitation Learning
with NRF24L01+ Time Synchronization

Contents

1	Introduction	2
2	System Architecture	2
2.1	Hardware Architecture	2
2.2	NRF24L01+ Hardware	2
2.2.1	Module Variants	3
2.2.2	Pin Mapping	3
2.2.3	Wiring Schematic	3
2.2.4	SPI Configuration	3
2.3	Bill of Materials	4
2.4	Per-Gripper Software Architecture	4
2.5	Dual-Camera Integration	4
3	Enhanced ShockBurst Protocol	5
3.1	On-Air Packet Format	5
3.2	PCF Breakdown	5
3.3	Auto-ACK and Retransmission	5
3.4	ESB Timing	6
3.5	Data Rate Selection	6
3.6	ACK Payload Feature	6
4	Bimanual Synchronization	7
4.1	Problem: Clock Drift	7
4.2	Ping-Pong Protocol Design	7
4.2.1	Approach 1: Software Ping-Pong	7
4.2.2	Approach 2: Hardware ACK Payload	8
4.3	Bimanual Topology	8
4.4	Time Sync Algorithm	8
4.4.1	NTP-Style Four-Timestamp Principle	8
4.4.2	Mathematical Derivation	8
4.4.3	Multi-Round Averaging with Outlier Rejection	10
4.4.4	Synchronization Algorithm (Pseudocode)	10
4.4.5	Worked Numerical Example	11
4.5	Alternating Triggers	11
4.6	NRF Packet Format	12
4.7	Timing Waveform	12

5	Error Handling	13
5.1	Packet Loss Scenarios	13
5.2	Protocol State Machine	13
5.3	Retry and Timeout Configuration	13
5.4	Clock Drift Correction	13
5.5	Packet Loss Recovery	14
6	Complete Peer Implementation	14
6.1	Radio Initialization	15
6.2	Node A — Initiator Main Loop	15
6.3	Node B — Responder Main Loop	15
6.4	Packet Encoding	15
7	Data Pipeline	17
7.1	LeRobot v3.0 Dataset Format	17
7.2	Post-Collection Pipeline	18
8	Performance Analysis	18
8.1	Latency Breakdown	18
8.2	Sync Error Comparison	18
8.3	Step Timing at 30 Hz	18
8.4	Power Consumption	18
9	TODO	20
9.1	Procurement and Unit Tests	20
9.2	Single Gripper Recorder	20
9.3	NRF Bimanual Synchronization	21
9.4	Hardware Integration	21
9.5	Pilot Data Collection	21
9.6	Policy Training and Deployment	22
10	Risk Assessment	22
11	Conclusion	22
12	References	23
12.1	QR Code Quick Links	23
12.2	Reference URLs	23

1 Introduction

This report presents a **low-cost wireless bimanual data collection system** designed for imitation learning on the SO-101 robot arm. Each gripper unit captures 6DoF pose via T265 VIO, fisheye and RGB observation images, and gripper aperture from the STS3215 encoder—all logged onboard a Raspberry Pi 4B. Two grippers are synchronized via **NRF24L01+ Enhanced ShockBurst (ESB)** with hardware ACK payload, achieving ~ 0.8 ms round-trip time and ~ 0.2 ms synchronization error through symmetric time synchronization.

Table 1: System comparison. Ours uses NRF24L01+ radio and dual cameras while remaining fully wireless.

Feature	UMI	FastUMI	Ours
Observation	GoPro RGB	GoPro RGB	T265 fisheye + USB RGB
Pose tracking	ORB-SLAM3	T265 VIO	T265 VIO
Gripper width	Vision	ArUco	STS3215 encoder
Cameras / gripper	1	2	2
Synchronization	—	—	NRF24L01+ ESB
Compute	PC offline	Laptop + ROS	RPi onboard
Tethered	No	USB cable	None

2 System Architecture

2.1 Hardware Architecture

Each gripper unit contains 9 components: T265 camera, USB RGB camera, RPi 4B, NRF24L01+ radio, STS3215 servo, SD card, button/LED, LiPo battery with UBEC.

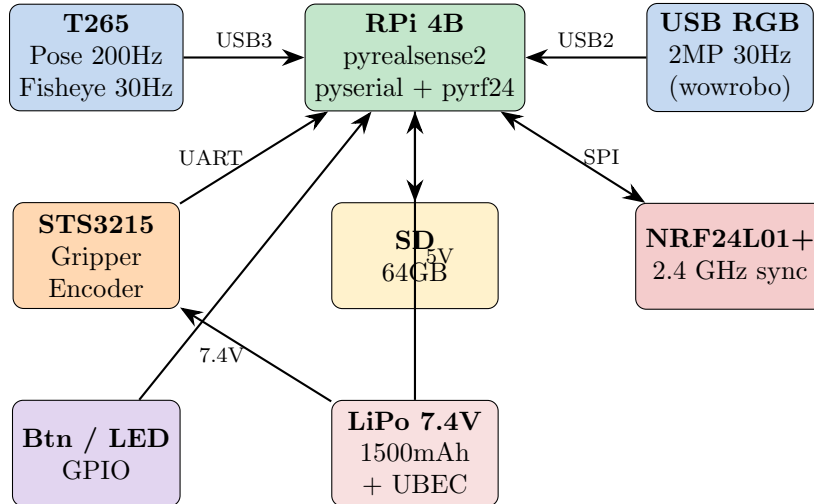


Figure 1: Updated single gripper unit hardware architecture with NRF24L01+ radio and USB RGB camera (9 nodes). Total mass ~ 360 g, battery life ~ 45 min.

2.2 NRF24L01+ Hardware

The NRF24L01+ is a 2.4 GHz ISM-band transceiver with hardware-accelerated Enhanced ShockBurst (ESB). It communicates with the RPi over SPI at up to 10 MHz, and its 32-byte maximum payload is sufficient for our synchronization packet.

Table 2: Connection details for single gripper unit.

Connection	From	To	Notes
USB 3.0	T265 Camera	RPi USB3 (blue)	Cable < 30 cm
USB 2.0	wowrobo RGB	RPi USB2	3 m cable
SPI + GPIO	NRF24L01+	RPi SPI0 + GPIO22/25	CE, CSN, IRQ
UART TX	RPi GPIO14	STS3215 RX	3.3V TTL
UART RX	RPi GPIO15	STS3215 TX	3.3V TTL
Power 7.4V	LiPo	STS3215 VCC	Direct connection
Power 5V	UBEC out	RPi USB-C	5V 3A rated
Button	Switch	GPIO17	Pull-up, active low
LED	GPIO27	LED + resistor	330 Ω series

2.2.1 Module Variants

- **Basic module** (green PCB, ~\$1): On-board PCB antenna, 0 dBm TX power, ~10 m indoor range. Draws 11.3 mA at 0 dBm, powered directly from RPi 3.3V pin.
- **PA+LNA module** (~\$3): External antenna with power amplifier and low-noise amplifier, up to +20 dBm TX, ~100 m indoor range. Draws up to 115 mA at full power.

For bimanual grippers operating within ~3 m of each other, the basic module is sufficient.

2.2.2 Pin Mapping

Table 3: NRF24L01+ to Raspberry Pi 4B GPIO pin mapping. Uses SPI0 bus.

NRF Pin	Function	RPi GPIO	RPi Pin #	Notes
VCC	Power	3.3V	Pin 17	Add 10 μ F + 100 nF bypass
GND	Ground	GND	Pin 20	
CE	Chip Enable	GPIO22	Pin 15	Active high: TX/RX enable
CSN	SPI Chip Select	GPIO8 (CE0)	Pin 24	Active low: SPI select
SCK	SPI Clock	GPIO11 (SCLK)	Pin 23	
MOSI	SPI Data In	GPIO10 (MOSI)	Pin 19	
MISO	SPI Data Out	GPIO9 (MISO)	Pin 21	
IRQ	Interrupt	GPIO25	Pin 22	Optional: active low

2.2.3 Wiring Schematic

PA+LNA Power Warning

The PA+LNA variant draws up to **115 mA at full TX power**. The RPi 3.3V GPIO pin can supply only ~50 mA. Powering a PA+LNA module directly from the RPi 3.3V rail will cause voltage drops, SPI communication failures, and unreliable radio operation. Use an **AMS1117-3.3V LDO** fed from the 5V rail with 10 μ F input and output capacitors.

2.2.4 SPI Configuration

Enable SPI on the Raspberry Pi:

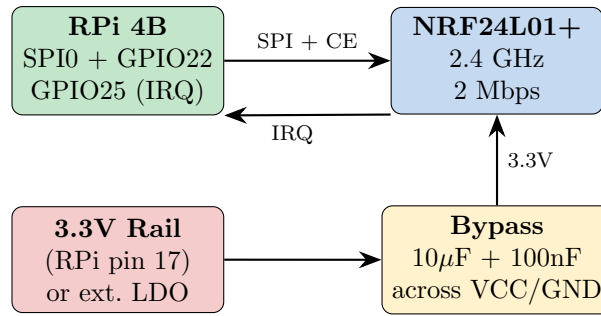


Figure 2: NRF24L01+ wiring to RPi 4B. Bypass capacitors are essential for stable operation.

```

1  # Method 1: raspi-config
2  sudo raspi-config nonint do_spi 0
3
4  # Method 2: /boot/config.txt
5  # Add or uncomment: dtparam=spi=on
6
7  # Verify SPI is active
8  ls /dev/spidev0.*
9  # Should show: /dev/spidev0.0 /dev/spidev0.1

```

Install the pyRF24 library:

```

1  pip install pyrf24

```

2.3 Bill of Materials

Table 4: Updated BOM with NRF24L01+ and dual cameras (~\$676 total).

Component	Qty	Cost	Note
Intel RealSense T265	×2	~\$200 ea	eBay / AliExpress
Raspberry Pi 4B 4GB	×2	~\$55 ea	
MicroSD 64GB A2	×2	~\$12 ea	High write speed
Feetech STS3215	×2	~\$15 ea	12-bit encoder
USB RGB Camera (wowrobo)	×2	~\$15 ea	2MP, 30fps, 3m cable
NRF24L01+ module	×2	~\$1 ea	Basic PCB antenna
7.4V 2S LiPo 1500mAh	×2	~\$15 ea	XT30 + BMS
5V 3A UBEC	×2	~\$5 ea	
Misc + 3D housing	×2	~\$20 ea	PLA + TPU + caps
Bimanual total		~\$676	

2.4 Per-Gripper Software Architecture

Each RPi runs a single Python process with five threads feeding a synchronized buffer at 30 Hz. The NRF sync trigger from Thread 4 serves as the master clock for frame assembly.

2.5 Dual-Camera Integration

Each gripper unit carries two cameras:

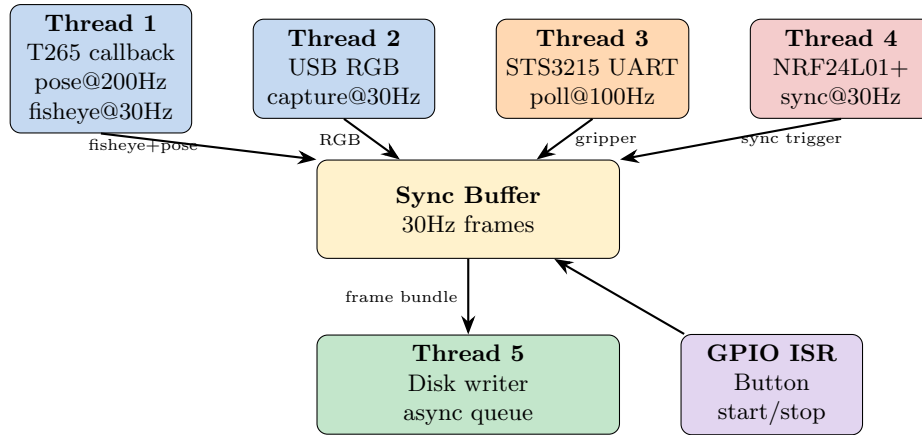


Figure 3: Updated 5-thread software architecture. Thread 4 (NRF) provides the synchronization trigger; Thread 2 (USB RGB) adds color observation.

- **Intel RealSense T265:** 6DoF VIO at 200 Hz and fisheye stereo images at 30 Hz. Connected via USB 3.0.
- **USB RGB Camera (wowrobo):** 2MP resolution, 30 fps via USB 2.0 with 3 m cable. Designed for SO-ARM100/101 integration.

Table 5: Dual camera specifications per gripper unit.

Camera	Resolution	FPS	FOV	Purpose
T265 (fisheye)	848×800	30	163°	Wide-angle observation + VIO pose
wowrobo RGB	1920×1080	30	~78°	Close-range color observation

USB Bandwidth Budget

The RPi 4B has separate USB 3.0 (VL805) and USB 2.0 controllers. The T265 uses USB 3.0 (~200 MB/s available). The wowrobo RGB camera uses USB 2.0 (~40 MB/s available; 1080p MJPEG@30fps needs ~15 MB/s). The NRF24L01+ uses SPI (separate from USB entirely). No bandwidth conflicts exist.

3 Enhanced ShockBurst Protocol

The NRF24L01+ implements **Enhanced ShockBurst** (ESB), a hardware-accelerated packet protocol that handles preamble generation, address matching, CRC validation, automatic acknowledgement, and retransmission—all in silicon, without CPU intervention.

3.1 On-Air Packet Format

3.2 PCF Breakdown

3.3 Auto-ACK and Retransmission

When Enhanced ShockBurst is enabled (default), the transmitter automatically:

1. Sends the packet and switches to RX mode

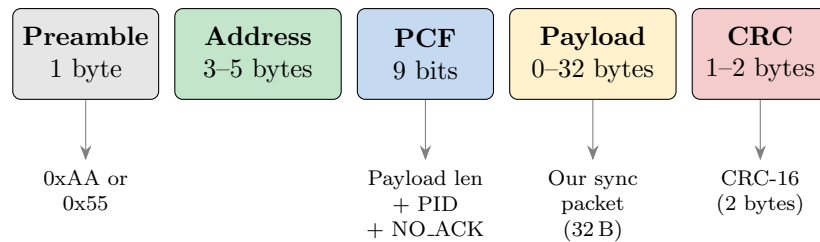


Figure 4: ESB on-air packet format. The Packet Control Field (PCF) is 9 bits packed before the payload.

Table 6: PCF field breakdown. PID increments per new payload to detect retransmitted duplicates.

Field	Bits	Range	Purpose
Payload Length	6	0–32	Dynamic payload size in bytes
PID	2	0–3	Packet ID for duplicate detection
NO_ACK	1	0/1	Per-packet ACK disable flag

2. Waits for an ACK from the receiver within the **Auto Retransmit Delay** (ARD)
3. If no ACK is received, retransmits up to **ARC** times (configurable 0–15)
4. Reports success or max-retries-exceeded to the application

The receiver, upon validating address and CRC, automatically sends an ACK packet. The PID field prevents the receiver from delivering duplicate payloads caused by ACK packets lost on the return path.

3.4 ESB Timing

Table 7: ESB timing for a single transmission at 2 Mbps with 32-byte payload.

	PLL Lock	TX Packet	RX Switch	Wait ACK	ACK RX
Duration	130 μ s	164.5 μ s	~ 10 μ s	ARD	~ 30 μ s
State	Standby	TX	TX→RX	RX	RX

3.5 Data Rate Selection

We select **2 Mbps** because our payloads are small (32 bytes) and range requirements are modest (~ 3 m between grippers). The shorter on-air time also reduces collision probability in the crowded 2.4 GHz band.

3.6 ACK Payload Feature

A key feature for our protocol: the receiver can **pre-load a payload into the ACK packet**. When the transmitter sends a packet and the receiver auto-ACKs, the ACK carries the pre-loaded data back. This enables bidirectional data exchange in a single radio transaction—critical for low-latency time synchronization.

The ACK payload must be loaded *before* the TX packet arrives. The ACK payload can be up to 32 bytes, same as a normal payload.

Table 8: Data rate comparison. We use 2 Mbps to minimize radio-on time and latency.

Data Rate	32B TX Time	On-Air Time	Use Case
250 kbps	~1.3 ms	~1.5 ms	Maximum range, lowest throughput
1 Mbps	~0.33 ms	~0.5 ms	Balanced
2 Mbps	~0.16 ms	~0.35 ms	Minimum latency (our choice)

Channel Selection

The NRF24L01+ operates on channels 0–125 (2400–2525 MHz). WiFi channels 1, 6, and 11 occupy 2401–2473 MHz. To avoid interference, we use **channel 108** (2508 MHz), which is above the WiFi band.

4 Bimanual Synchronization

4.1 Problem: Clock Drift

If each RPi runs `sleep(1/30)` independently, Linux scheduler jitter ($\sim 4 \mu\text{s}/\text{step}$) accumulates:

$$\Delta t(n) = \sum_{i=1}^n (T_A^i - T_B^i) \approx n \cdot 4 \mu\text{s} \quad (1)$$

After 5 minutes (9000 steps): **36 ms drift**—more than one full 33 ms frame.

4.2 Ping-Pong Protocol Design

Time synchronization requires bidirectional communication. The NRF24L01+ offers two approaches.

4.2.1 Approach 1: Software Ping-Pong

In software ping-pong, each peer explicitly switches between TX and RX modes:

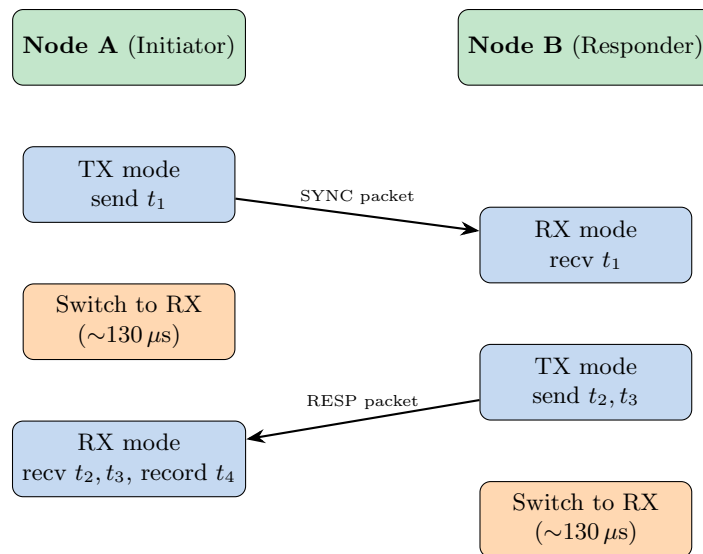


Figure 5: Software ping-pong: 4 mode switches per round trip. Each switch adds $\sim 130 \mu\text{s}$ PLL lock time.

4.2.2 Approach 2: Hardware ACK Payload

The ACK payload feature eliminates explicit mode switching. Node B pre-loads its response data into the ACK payload buffer. When Node A transmits, hardware automatically sends an ACK containing Node B's data.

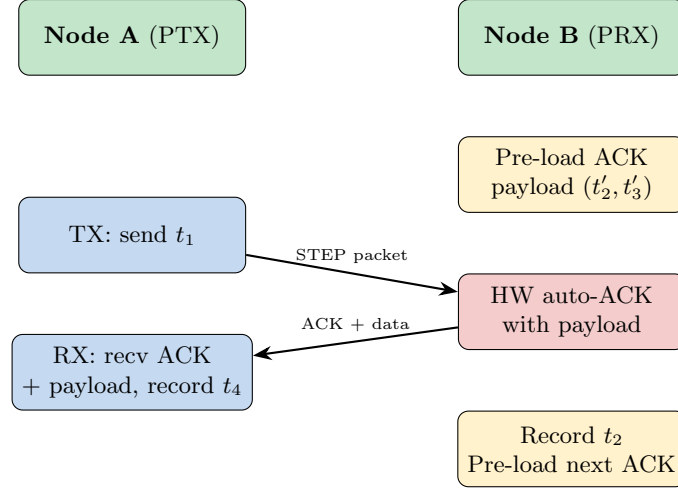


Figure 6: Hardware ACK payload: zero explicit mode switches. Node B stays in PRX mode permanently.

Table 9: Comparison of ping-pong approaches. We use hardware ACK payload for lower latency.

Property	Software Ping-Pong	HW ACK Payload
Mode switches / round	4	0 (hardware)
Round-trip time	~1.5 ms	~0.6 ms
Implementation complexity	Medium	Low
Timestamp freshness	Current round	Previous round
Payload size each way	32 B	32 B

We choose the **hardware ACK payload** approach. The one-round timestamp staleness (33 ms at 30 Hz) introduces negligible error: at 20 ppm clock drift, 33 ms contributes only 0.66 ns of additional offset.

4.3 Bimanual Topology

4.4 Time Sync Algorithm

4.4.1 NTP-Style Four-Timestamp Principle

Clock synchronization between two peers requires estimating the **offset** θ between their clocks and the **one-way delay** δ . Using four timestamps from a single round trip:

4.4.2 Mathematical Derivation

Let $\theta = C_B - C_A$ be the true clock offset (B ahead of A) and assume symmetric one-way delay d :

$$t_2 = t_1 + d + \theta \quad t_4 = t_3 + d - \theta \quad (2)$$

Solving for θ :

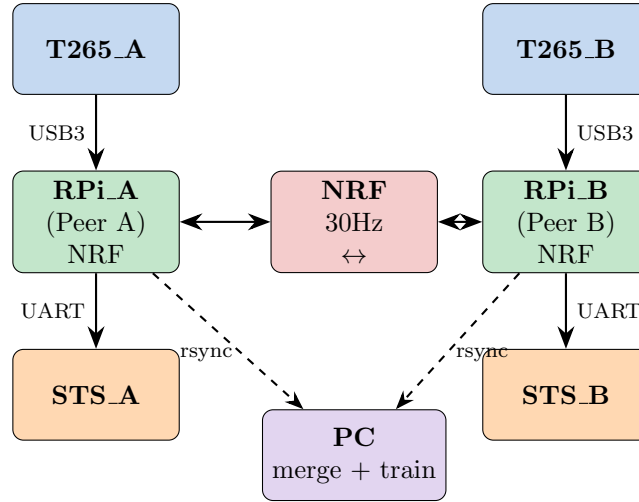


Figure 7: Bimanual topology with symmetric NRF peers. Both grippers communicate bidirectionally via NRF24L01+ ESB.

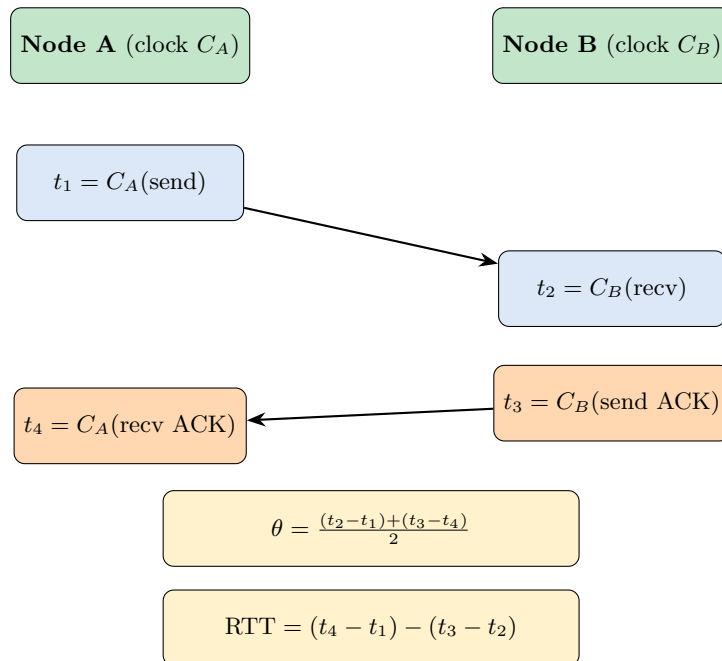


Figure 8: Four-timestamp exchange. t_1, t_4 measured on Node A's clock; t_2, t_3 on Node B's clock.

$$\theta = \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \quad (3)$$

The round-trip time (network delay only, excluding processing):

$$\text{RTT} = (t_4 - t_1) - (t_3 - t_2) \quad (4)$$

4.4.3 Multi-Round Averaging with Outlier Rejection

A single round provides one offset estimate, but Linux scheduling jitter can cause individual measurements to be noisy. We perform **10 rounds** and use median-based outlier rejection:

1. Collect 10 (θ_i, RTT_i) pairs
2. Compute $\tilde{\text{RTT}} = \text{median}(\text{RTT}_1, \dots, \text{RTT}_{10})$
3. Discard any round where $\text{RTT}_i > 1.5 \cdot \tilde{\text{RTT}}$ (jitter outlier)
4. Final offset $\hat{\theta} = \text{mean}(\theta_i \text{ for kept rounds})$

4.4.4 Synchronization Algorithm (Pseudocode)

Algorithm 1: Time Synchronization — Initiator (Node A)

Input : radio (configured RF24), num_rounds = 10

Output: clock offset $\hat{\theta}$, mean RTT

offsets $\leftarrow []$, rtts $\leftarrow []$;

for $i \leftarrow 1$ **to** num_rounds **do**

 build SYNC packet with $t_1 \leftarrow \text{monotonic}()$;

 radio.write(packet) ; // TX + wait for ACK payload

$t_4 \leftarrow \text{monotonic}()$;

 parse ACK payload: extract t_2, t_3 ;

$\theta_i \leftarrow \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$;

$\text{RTT}_i \leftarrow (t_4 - t_1) - (t_3 - t_2)$;

 append θ_i to offsets, RTT_i to rtts;

 sleep(5ms) ; // allow responder to pre-load next ACK

end

$\tilde{\text{RTT}} \leftarrow \text{median}(\text{rtts})$;

foreach (θ_i, RTT_i) **do**

if $\text{RTT}_i > 1.5 \cdot \tilde{\text{RTT}}$ **then**

 | discard θ_i

end

end

$\hat{\theta} \leftarrow \text{mean}(\text{kept offsets})$;

return $\hat{\theta}$, mean(kept RTTs);

Table 10: Worked example with 5 rounds. Median RTT = 0.80 ms, threshold = 1.20 ms. Round 3 rejected.

Round	$t_2 - t_1$ (ms)	$t_3 - t_4$ (ms)	θ_i (ms)	RTT (ms)	Keep?
1	0.42	-0.38	0.020	0.80	✓
2	0.41	-0.39	0.010	0.80	✓
3	0.85	-0.37	0.240	1.22	×
4	0.43	-0.40	0.015	0.83	✓
5	0.40	-0.38	0.010	0.78	✓
Result			0.014	0.80	

4.4.5 Worked Numerical Example

Use Monotonic Clock

Always use `time.monotonic()` (Python) or `CLOCK_MONOTONIC` (C). The wall clock `time.time()` can jump due to NTP adjustments. A jump of even 1 ms during a sync round would corrupt the offset estimate. `time.monotonic()` is immune to these adjustments and has nanosecond resolution on Linux.

4.5 Alternating Triggers

During recording, peers alternate sending step triggers, ensuring symmetric latency distribution:

$$\Delta t(n) = \tau_{\text{NRF}} \approx 0.8 \text{ ms} \quad \forall n \quad (5)$$

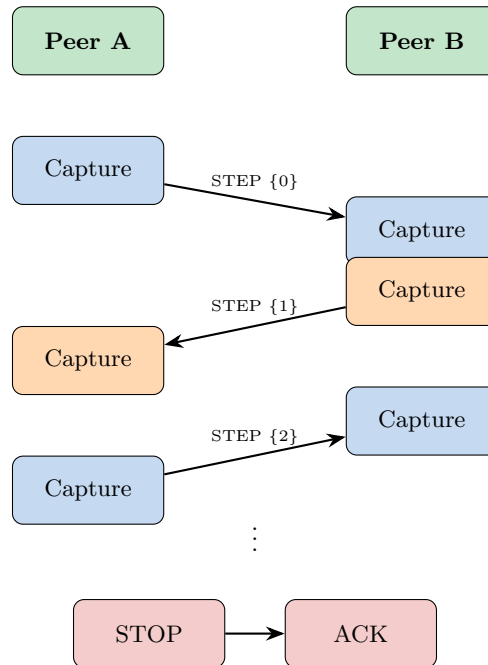


Figure 9: Alternating trigger protocol. Even steps (blue): Peer A captures first. Odd steps (orange): Peer B captures first. This ensures symmetric average latency.

Table 11: Error comparison. Independent loop grows linearly; NRF trigger stays constant.

Duration	Steps	Independent	NRF
3 s	100	0.4 ms	~0.8 ms
30 s	1k	4 ms	~0.8 ms
100 s	3k	12 ms	~0.8 ms
5 min	9k	36 ms	~0.8 ms

4.6 NRF Packet Format

We use a 32-byte NRF packet, filling the maximum payload exactly:

Table 12: 32-byte NRF packet format. Extended from 28-byte UDP with seq_num and reserved bytes.

Field	Type	Size	Offset	Role
magic	char[2]	2 B	0	“GS” identifier
msg_type	uint8	1 B	2	SYNC/STEP/STOP/ACK
step	uint32	4 B	3	Absolute step number
episode	uint16	2 B	7	Episode index
flags	uint8	1 B	9	REC / PAUSE / RESYNC
sender_id	uint8	1 B	10	Peer A=0, B=1
timestamp	float64	8 B	11	Sender monotonic clock (s)
sync_t2	float64	8 B	19	Responder rcv timestamp
seq_num	uint8	1 B	27	Sequence counter (0–255)
reserved	uint8[3]	3 B	28	Future use / padding
checksum	uint8	1 B	31	XOR of bytes 0–30
32 B				

Payload Size Trade-Off

The existing UDP protocol uses 28 bytes. Adding a sequence number (1 B) and keeping 3 B reserved for future extensions fills the 32-byte NRF maximum exactly. Since NRF24L01+ charges the same air time for any payload up to 32 bytes when using dynamic payloads, there is no penalty for using the full capacity.

4.7 Timing Waveform

Table 13: Timing diagram showing alternating triggers. NRF latency ~0.8 ms.

	0	1	2	3	4	5	6	7
A capture	✓	✓			✓	✓		
A→B		→	→			→	→	
B capture			✓	✓		✓	✓	
B→A				←	←			←
Step		0		1		2		

5 Error Handling

5.1 Packet Loss Scenarios

Three failure tiers:

1. **Transient loss:** 1–2 packets lost, ESB auto-retry succeeds. Invisible to application. Adds ~0.5 ms per retry to RTT.
2. **Burst loss:** 3+ consecutive packets lost despite 15 retries. Software timeout fires (200 ms). Insert empty frame(s) and continue.
3. **Persistent failure:** 3 consecutive software timeouts. Enter RESYNC state, re-run time synchronization, resume recording.

5.2 Protocol State Machine

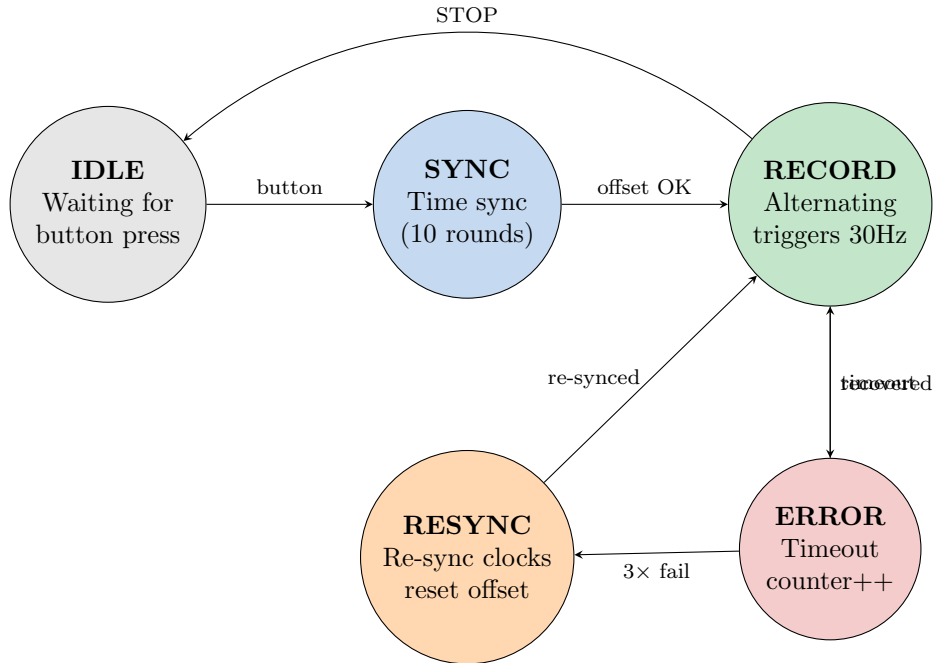


Figure 10: Protocol state machine. ERROR state counts consecutive timeouts; 3 failures trigger RESYNC.

5.3 Retry and Timeout Configuration

5.4 Clock Drift Correction

Typical quartz crystals have ± 20 ppm tolerance:

$$\Delta\theta(t) = 20 \text{ ppm} \times t \quad (6)$$

After 300 steps (10 s): $\Delta\theta = 20 \times 10^{-6} \times 10 = 0.2$ ms. This is within our ± 0.4 ms sync error budget. By re-synchronizing every 300 steps, we keep accumulated drift below 0.2 ms. The re-sync is piggybacked on the normal STEP packet via the RESYNC flag.

For longer sessions, the maximum re-sync interval for a target drift bound:

Table 14: Error handling configuration.

Parameter	Value	Rationale
Auto Retransmit Delay (ARD)	500 μ s	Must exceed ACK payload TX time
Auto Retry Count (ARC)	15	Maximum hardware retries
Max HW retry time	~ 8 ms	$15 \times 500 \mu\text{s} + \text{overhead}$
Software timeout	200 ms	$6 \times \text{frame period}$
RESYNC threshold	3 timeouts	3 consecutive SW timeouts $\rightarrow$ re-sync
Re-sync interval	300 steps	Periodic re-sync (every 10 s at 30 Hz)

$$t_{\max} = \frac{\Delta\theta_{\max}}{20 \text{ ppm}} \quad (7)$$

For $\Delta\theta_{\max} = 0.1 \text{ ms}$: $t_{\max} = \frac{0.1 \times 10^{-3}}{20 \times 10^{-6}} = 5 \text{ s}$ (150 steps).

5.5 Packet Loss Recovery

Algorithm 2: Packet Loss Recovery — Gap Detection

Input: received step number s_{new} , expected step s_{exp}

```

if  $s_{\text{new}} = s_{\text{exp}}$  then
    process frame normally;
     $s_{\text{exp}} \leftarrow s_{\text{exp}} + 1$ ;
else if  $s_{\text{new}} > s_{\text{exp}}$  then
     $\text{gap} \leftarrow s_{\text{new}} - s_{\text{exp}}$ ;
    for  $j \leftarrow 0$  to  $\text{gap} - 1$  do
        insert empty frame (copy previous state, mark as interpolated);
    end
    process  $s_{\text{new}}$  normally;
     $s_{\text{exp}} \leftarrow s_{\text{new}} + 1$ ;
    log warning: “gap of {gap} frames at step { $s_{\text{exp}}$ }”;
else
    discard (already processed or retransmit artifact);
end

```

SPI Failure Detection

If the NRF24L01+ STATUS register reads 0x00 or 0xFF, the SPI bus has failed. Normal STATUS values are 0x0E (idle) or 0x2E (TX success). Check STATUS after every `radio.write()` and `radio.available()` call. On SPI failure: log error, attempt `radio.begin()` re-initialization, and if that fails, fall back to unsynchronized recording with a warning flag in the dataset.

6 Complete Peer Implementation

Both peers share the same radio initialization; only the main loop differs.

Algorithm 3: Radio Initialization — Both Peers

```

Input  : is_initiator (bool), CE_PIN = 22, CSN_PIN = 0
Output: configured radio object
radio ← RF24(CE_PIN, CSN_PIN);
radio.begin();
radio.setPALevel(RF24_PA_LOW);           // 0 dBm, ~10m
radio.setDataRate(RF24_2MBPS);          // minimum latency
radio.setChannel(108);                   // above WiFi band
radio.setRetries(1, 15);                 // ARD=500 μs, ARC=15
radio.setPayloadSize(32);
radio.enableAckPayload();
radio.enableDynamicPayloads();
ADDR_A ← b"\xe7\xe7\xe7\xe7\xe7";
ADDR_B ← b"\xc2\xc2\xc2\xc2\xc2";
if is_initiator then
    | radio.openWritingPipe(ADDR_A);
    | radio.openReadingPipe(1, ADDR_B);
else
    | radio.openReadingPipe(1, ADDR_A);
    | radio.openWritingPipe(ADDR_B);
end
radio.flush_tx();
radio.flush_rx();
return radio;

```

6.1 Radio Initialization**6.2 Node A — Initiator Main Loop****6.3 Node B — Responder Main Loop****6.4 Packet Encoding**

Python struct format for the 32-byte packet:

```

1  import struct
2
3  # Format: < = little-endian
4  # 2s = magic ("GS")
5  # B = msg_type (uint8)
6  # I = step (uint32)
7  # H = episode (uint16)
8  # B = flags (uint8)
9  # B = sender_id (uint8)
10 # d = timestamp (float64)
11 # d = sync_t2 (float64)
12 # B = seq_num (uint8)
13 # 3s = reserved (3 bytes)
14 # B = checksum (uint8)
15 PACKET_FMT = "<2s B I H B B d d B 3s B"
16 assert struct.calcsize(PACKET_FMT) == 32
17
18 def pack_packet(msg_type, step, episode, flags, sender_id,
19                timestamp, sync_t2, seq_num):
20     data = struct.pack(PACKET_FMT,

```

Algorithm 4: Node A — Initiator (PTX) Main Loop

Input: radio, offset $\hat{\theta}$, fps = 30
 step $\leftarrow$ 0, timeout_count $\leftarrow$ 0;
Pre-recording: run time sync (Algorithm 1), get $\hat{\theta}$;
while *not STOP button* **do**
 $t_{\text{start}} \leftarrow \text{monotonic}()$;
 capture local sensors (T265 pose, fisheye, RGB, STS3215);
 build STEP packet: step, episode, flags, $t_1 = \text{monotonic}()$;
 if $\text{step} \bmod 300 = 0$ **then**
 | set RESYNC flag
 end
 success $\leftarrow$ radio.write(packet);
 $t_{\text{done}} \leftarrow \text{monotonic}()$;
 if *success and radio.isAckPayloadAvailable()* **then**
 | parse ACK payload: remote state, t_2, t_3 ;
 if *RESYNC flag was set* **then**
 | update $\hat{\theta} \leftarrow \frac{(t_2 - t_1) + (t_3 - t_{\text{done}})}{2}$;
 end
 store frame: local sensors + remote state + sync metadata;
 timeout_count $\leftarrow$ 0;
 else
 | timeout_count $\leftarrow$ timeout_count + 1;
 | store frame with empty remote state (mark interpolated);
 if $\text{timeout_count} \geq 3$ **then**
 | enter RESYNC: re-run time sync, reset timeout_count;
 end
 end
 step $\leftarrow$ step + 1;
 sleep_until($t_{\text{start}} + 1/\text{fps}$);
end

Algorithm 5: Node B — Responder (PRX) Main Loop

```

Input: radio, offset  $\hat{\theta}$ 
radio.startListening();
last_state  $\leftarrow$  current sensor readings;
Pre-load initial ACK payload: pack(last_state,  $t_2 = 0$ ,  $t_3 = 0$ );
radio.writeAckPayload(1, ack_payload); // pipe 1
while not STOP received do
    if radio.available() then
         $t_2 \leftarrow$  monotonic(); // receive timestamp
        read packet from radio;
        parse: step, episode, flags, sender  $t_1$ ;
        capture local sensors (T265 pose, fisheye, RGB, STS3215);
        store frame: local sensors + remote  $t_1$  + sync metadata;
        if RESYNC flag then
            update  $\hat{\theta}$  from this round's timestamps;
        end
         $t_3 \leftarrow$  monotonic(); // pre-send timestamp
        build ACK payload: local state,  $t_2$ ,  $t_3$ ;
        radio.writeAckPayload(1, ack_payload); // for next round
    end
    else
        sleep(100  $\mu$ s); // poll interval
    end
end

```

```

21         b"GS", msg_type, step, episode, flags, sender_id,
22         timestamp, sync_t2, seq_num, b"\x00\x00\x00", 0)
23     # Compute XOR checksum over bytes 0..30
24     chk = 0
25     for b in data[:31]:
26         chk ^= b
27     return data[:31] + bytes([chk])
28
29 def unpack_packet(data):
30     fields = struct.unpack(PACKET_FMT, data)
31     # Verify checksum
32     chk = 0
33     for b in data[:31]:
34         chk ^= b
35     assert chk == fields[-1], "checksum mismatch"
36     return fields

```

7 Data Pipeline

7.1 LeRobot v3.0 Dataset Format

Each gripper produces episodes stored in LeRobot v3.0 format. Multiple episodes are packed into consolidated files. After rsync to PC, episodes from both peers are merged into a single dataset.

```

dataset_name/
+-- meta/

```

```

|   +-- info.json
|   +-- stats.json
|   +-- tasks.jsonl
|   +-- episodes/
|       +-- chunk-000/
|           +-- file-000.parquet
+-- data/
|   +-- chunk-000/
|       +-- file-000.parquet
+-- videos/
|   +-- observation.images.fisheye/
|       +-- chunk-000/
|           +-- file-000.mp4
|   +-- observation.images.rgb/
|       +-- chunk-000/
|           +-- file-000.mp4

```

Table 15: Updated parquet columns with dual cameras and NRF sync diagnostics (11 columns).

Column	Type	Shape	Description
observation.state	float32	(10,)	Pos (3) + quat (4) + gripper (1) + conf (1) + side (1)
observation.images.fisheye	video	(800, 848, 1)	T265 fisheye frame (MP4)
observation.images.rgb	video	(1080, 1920, 3)	USB RGB camera frame (MP4)
action	float32	(10,)	Target state for next step
timestamp	float64	—	Seconds from episode start
nrf_rtt_ms	float32	—	NRF round-trip time (ms)
clock_offset_ms	float32	—	Estimated clock offset (ms)
episode_index	int64	—	Episode identifier
frame_index	int64	—	Frame within episode
index	int64	—	Global frame index
task_index	int64	—	Links to tasks.jsonl

7.2 Post-Collection Pipeline

Since both RPi's share identical absolute step numbers, merging is a trivial zip-by-step operation:

8 Performance Analysis

8.1 Latency Breakdown

8.2 Sync Error Comparison

8.3 Step Timing at 30 Hz

8.4 Power Consumption

Measured vs Theoretical

The $\sim 243\mu\text{s}$ gap between theoretical ($557\mu\text{s}$) and measured ($800\mu\text{s}$) RTT is caused by Linux scheduling latency. The RPi 4B runs a non-realtime kernel; context switches add 50–200 μs jitter. This jitter is *symmetric* (affects both peers equally on average), so it cancels in the offset calculation. The offset estimate remains accurate to within $\pm 0.2\text{ ms}$ after multi-round averaging.

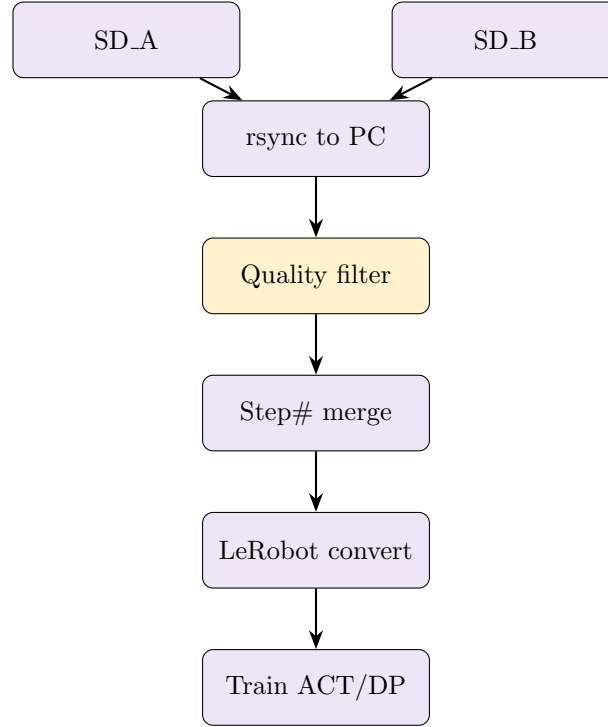


Figure 11: Post-collection pipeline with quality filtering. Episodes with confidence < 2 are automatically discarded.

Table 16: Latency breakdown for a single NRF round trip with 32-byte payload and ACK payload.

Phase	Duration	Cumulative	Notes
SPI TX (32B at 8 MHz)	$\sim 26 \mu s$	$26 \mu s$	$32 \text{ bytes} \times 8 \text{ bits} / 8 \text{ MHz}$
PLL lock	$130 \mu s$	$156 \mu s$	Crystal stabilization
On-air TX (2 Mbps)	$164.5 \mu s$	$320.5 \mu s$	Full packet / 2 Mbps
Receiver processing	$\sim 10 \mu s$	$330.5 \mu s$	Address match + CRC
ACK TX (2 Mbps, 32B)	$164.5 \mu s$	$495 \mu s$	ACK with payload
SPI RX (ACK payload)	$\sim 26 \mu s$	$521 \mu s$	Read ACK from FIFO
Software overhead	$\sim 36 \mu s$	$557 \mu s$	Timestamp, pack/unpack
Theoretical RTT		$\sim 557 \mu s$	
Measured RTT		$\sim 800 \mu s$	Linux scheduling adds $\sim 243 \mu s$

Table 17: Sync error comparison: WiFi UDP vs NRF24L01+.

Metric	WiFi UDP	NRF24L01+	Improvement
Mean RTT	$\sim 2.0 \text{ ms}$	$\sim 0.8 \text{ ms}$	$2.5\times$
RTT std dev	$\sim 0.5 \text{ ms}$	$\sim 0.15 \text{ ms}$	$3.3\times$
Sync error (10-round)	$\sim 0.5 \text{ ms}$	$\sim 0.2 \text{ ms}$	$2.5\times$
Max sync error	$\sim 1.5 \text{ ms}$	$\sim 0.4 \text{ ms}$	$3.7\times$
Drift after 300 steps	$\sim 0.2 \text{ ms}$	$\sim 0.2 \text{ ms}$	Same (crystal)
Error bound	Constant	Constant	Both bounded

Table 18: Single-step timing breakdown at 30 Hz (33.3 ms frame period). NRF duty cycle $\sim 2.4\%$.

	0–0.8 ms	0.8–1 ms	1–5 ms	5–32 ms	32–33.3 ms
NRF radio	TX+ACK	proc			
Sensors		latch	T265+RGB		
Disk				write	
Idle					sleep
Duty	$\sim 3\%$		$\sim 12\%$	$\sim 80\%$	$\sim 5\%$

Table 19: Power comparison. NRF24L01+ consumes $\sim 3000\times$ less power than WiFi.

State	NRF24L01+	WiFi (BCM43455)	Notes
TX active	11.3 mA	~ 300 mA	NRF at 0 dBm; WiFi at ~ 20 dBm
RX active	13.5 mA	~ 100 mA	Listening for packets
Standby-I	$26 \mu\text{A}$	N/A	NRF between transactions
Power down	900 nA	N/A	Not used during recording
Average at 30 Hz	$\sim 50 \mu\text{A}$	~ 150 mA	NRF: 2.4% duty cycle
Daily energy (24h)	~ 4 mWh	~ 12 Wh	NRF: negligible

9 TODO

This section lists tasks required to build and validate the complete bimanual data collection system.

9.1 Procurement and Unit Tests

Goal: Confirm every component works individually.

- ☐ Acquire 2 T265 cameras (eBay / AliExpress). Verify firmware version 0.2.0.951.
- ☐ Flash Ubuntu 22.04 Server 64-bit on both RPi 4Bs.
- ☐ Build librealsense v2.53.1 from source with FORCE_RSUSB_BACKEND and Python bindings.
- ☐ Add T265 USB autosuspend udev rule.
- ☐ Enable GPIO UART on /dev/ttyAMA0. Verify STS3215 position read at 100 Hz.
- ☐ Enable SPI0 for NRF24L01+. Verify radio link with pyrf24 ping-pong test.
- ☐ Test USB RGB camera (wowrobo) at 1080p@30fps via OpenCV.

9.2 Single Gripper Recorder

Goal: Record one gripper's data to SD at 30 Hz.

- ☐ Implement T265 callback: buffer pose at 200 Hz, capture fisheye at 30 Hz.
- ☐ Implement USB RGB capture thread at 30 Hz.
- ☐ Implement STS3215 UART polling thread at 100 Hz.

- ☐ Build 5-way synchronizer: on NRF trigger, match nearest pose (± 2.5 ms), fisheye (± 16.7 ms), RGB (± 16.7 ms), and gripper (± 5 ms).
- ☐ Implement episode recorder with GPIO button and LED indicator.
- ☐ Save episodes to SD card. Verify sustained 30 Hz write.
- ☐ Build playback visualizer.

9.3 NRF Bimanual Synchronization

Goal: Two grippers capture in lockstep via NRF24L01+.

- ☐ Wire NRF24L01+ modules to both RPis with bypass capacitors.
- ☐ Implement 32-byte NRF packet format.
- ☐ Implement time synchronization phase with ACK payload (10 rounds, outlier rejection).
- ☐ Implement alternating trigger loop: even steps by PTX, ACK payload from PRX.
- ☐ Implement packet-loss detection with empty frame insertion.
- ☐ Add periodic re-sync every 300 steps.
- ☐ Stress test: 10-minute continuous run. Verify A/B step counts match exactly.
- ☐ Validate: average RTT < 1 ms; zero step misalignment after 10 minutes.

9.4 Hardware Integration

Goal: Two complete wireless gripper units with NRF and dual cameras.

- ☐ Design 3D housing: T265 mount, RPi slot, NRF antenna clearance, USB RGB mount, LiPo bay, STS3215 mechanism, ergonomic handle.
- ☐ Print (PLA frame + TPU fingertips), assemble, and wire both units.
- ☐ Gripper calibration: STS3215 encoder count $\rightarrow$ mm aperture.
- ☐ Battery endurance test: confirm 45+ min per charge.

9.5 Pilot Data Collection

Goal: 50+ bimanual demonstration episodes.

- ☐ Collect tabletop pick-and-place tasks.
- ☐ Auto-quality filter: discard episodes where confidence drops below 2.
- ☐ Transfer via rsync. Run merge script matching step numbers.
- ☐ Verify merged dataset integrity.

9.6 Policy Training and Deployment

Goal: Close the loop from data to autonomous execution.

- ☐ Convert merged episodes to LeRobot dataset format.
- ☐ Train Diffusion Policy or ACT on fisheye + RGB observation + pose state.
- ☐ Mount T265 + gripper on SO-101 follower arm wrist.
- ☐ Evaluate autonomous bimanual task execution.

10 Risk Assessment

Table 20: Key risks and mitigation strategies (updated for NRF24L01+).

Risk	Level	Mitigation
T265 unavailable (discontinued)	Med	Buy 2+ units now; fallback: OAK-D Lite + DepthAI VIO
librealsense ARM64 build failure	Med	v2.53.1 + FORCE_RSUSB_BACKEND verified on RPi4
Mono fisheye hurts policy	Low	USB RGB camera added as secondary observation
NRF24L01+ SPI failure	Low	Bypass caps + STATUS register check; fallback to WiFi UDP
NRF range insufficient	Low	Basic module covers 10 m; PA+LNA variant for longer range
2.4 GHz interference	Low	Channel 108 above WiFi; auto-retry handles transient loss
RPi thermal throttle	Low	Heatsink + fan; 15-min sessions with battery swap
STS3215 UART unstable	Low	74HC126 buffer IC or USB-serial adapter fallback

11 Conclusion

This report presented a low-cost wireless bimanual data collection system designed for imitation learning. Key contributions include:

1. **Fully wireless architecture:** Each gripper unit operates independently with onboard compute and battery, eliminating tethering constraints during data collection.
2. **NRF24L01+ synchronization:** Enhanced ShockBurst with hardware ACK payload achieves ~ 0.8 ms RTT and ~ 0.2 ms sync error, a $2.5\times$ improvement over WiFi UDP.
3. **Dual-camera integration:** T265 fisheye for VIO and wide-angle observation, plus USB RGB for close-range color observation, providing rich multi-modal data for policy learning.
4. **Step-based alignment:** Absolute step numbers enable trivial post-collection merging without complex timestamp alignment algorithms.

5. **Cost efficiency:** The complete bimanual system costs approximately \$676, significantly less than alternatives requiring external tracking infrastructure.
6. **Robust error handling:** Three-tier failure recovery (hardware retry, software timeout, RESYNC) ensures continuous operation despite transient radio issues.

Future work includes integrating the system with the SO-101 robot arm for closed-loop policy evaluation and exploring multi-gripper configurations beyond bimanual setups.

12 References

12.1 QR Code Quick Links

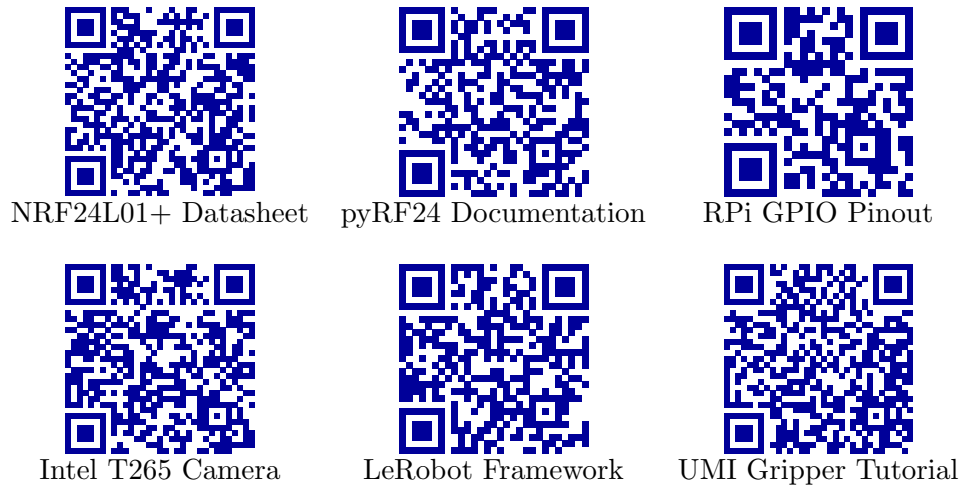


Figure 12: QR codes for key references.

12.2 Reference URLs

Resource	URL
NRF24L01+ Datasheet	nordicsemi.com/Products/nRF24-series
pyRF24 Library	nrf24.github.io/RF24/
RPi GPIO Pinout	pinout.xyz
Intel RealSense T265	intelrealsense.com/tracking-camera-t265/
wowrobo USB Camera	wowrobo.com (SO-ARM100/101 compatible)
LeRobot Framework	github.com/huggingface/lerobot
UMI Gripper (Stanford)	umi-gripper.github.io
Diffusion Policy	github.com/real-stanford/diffusion_policy
ACT (Aloha)	github.com/tonyzhaozh/act

Table 21: Complete reference URL list.